



Documentación del proyecto — Study Tracker

Guía completa para entender la arquitectura, el stack y **qué hace cada archivo** del proyecto. Pensada para que cualquier persona (o tú mismo en el futuro) pueda situarse rápido.

1. ¿Qué es Study Tracker?

Aplicación web para **registrar y analizar sesiones de estudio** con un temporizador Pomodoro configurable. Cada usuario tiene su cuenta, ve solo sus datos, personaliza la apariencia y consulta estadísticas avanzadas (rachas, regularidad, heatmap de consistencia, radar temático, etc.).

Es un proyecto **full-stack minimalista**: un backend Flask + SQLite y un frontend de dos archivos estáticos, sin frameworks de JavaScript ni proceso de build.

2. Stack tecnológico

Backend

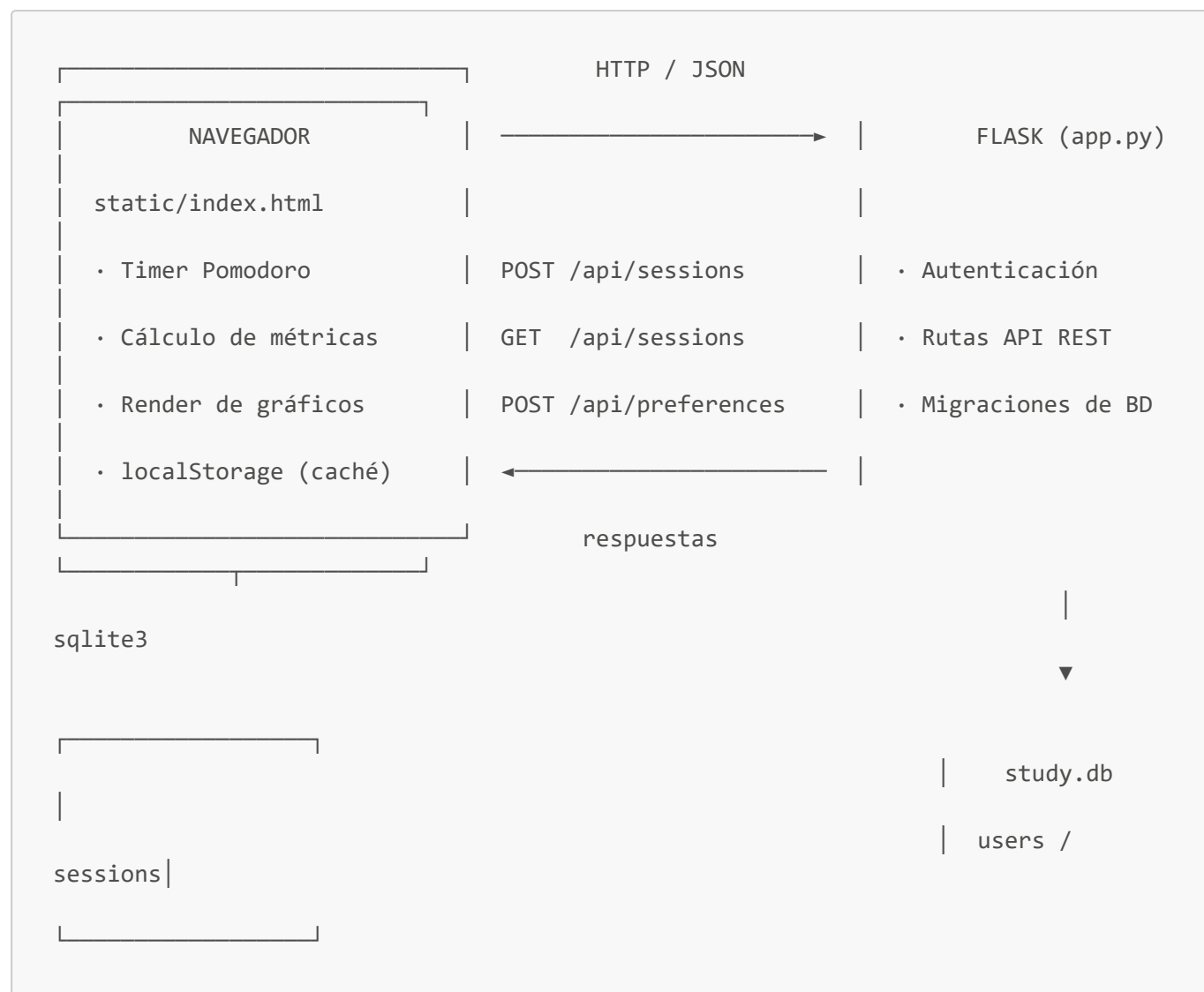
Tecnología	Rol
Python 3	Lenguaje del servidor
Flask	Framework web: rutas, API REST, servir archivos estáticos
Flask-Login	Sesiones de usuario, protección de rutas (<code>@login_required</code>)
Werkzeug	Hash seguro de contraseñas (<code>generate_password_hash</code> / <code>check_password_hash</code>)
SQLite 3	Base de datos embebida (módulo <code>sqlite3</code> de la stdlib, sin servidor aparte)
WSGI	Interfaz de despliegue (para PythonAnywhere)

Frontend

Tecnología	Rol
HTML5 / CSS3	Estructura y estilos (glassmorphism, variables CSS, 4 temas)
JavaScript (ES6+) vanilla	Toda la lógica de cliente, sin framework
Chart.js 4.4 (CDN)	Gráficos: línea, barras, radar
localStorage	Caché local del historial + sincronización con el backend
Tabler Icons (CDN)	Iconografía
Google Fonts — Inter (CDN)	Tipografía

No hay Node/npm/bundler en producción. El frontend son archivos estáticos que Flask entrega tal cual. Las únicas dependencias externas se cargan por CDN.

3. Arquitectura general



Punto clave — doble almacenamiento sincronizado:

- El cliente guarda las sesiones en **localStorage** (respuesta instantánea, funciona offline).
- Al iniciar, `syncSessions()` descarga el historial real del backend y hace un **merge idempotente por ts** (timestamp), evitando duplicados y **subiendo** al backend los registros que solo existían localmente (*backfill*).
- Así los datos **no se pierden entre dispositivos** y el backend es la fuente de verdad.

Las **estadísticas se calculan en el cliente** a partir de ese historial ya sincronizado (existe también `/api/stats` como cálculo alternativo en el servidor, hoy no usado por la UI).

4. Estructura de carpetas

```
study_tracker/
├── app.py           ← Backend Flask: rutas, API, autenticación, migraciones
├── wsgi.py          ← Punto de entrada WSGI (despliegue en PythonAnywhere)
├── ver_db.py         ← Script de utilidad para inspeccionar la base de datos
└── requirements.txt ← Dependencias de Python
```

— study.db	← Base de datos SQLite (se crea/migra automáticamente)
— .secret_key	← Clave de sesión generada (NO subir a git)
— README.md	← Guía rápida de instalación y API
— DOCUMENTACION.md	← Este documento
— static/	
— index.html	← Aplicación completa (UI + CSS + JS)
— login.html	← Pantalla de registro / inicio de sesión
— venv/	← Entorno virtual de Python (dependencias instaladas)
— __pycache__/	← Caché de bytecode de Python (autogenerado)

5. Explicación archivo por archivo

 **app.py** — El corazón del backend (~325 líneas)

Contiene toda la lógica del servidor. Se organiza en bloques:

a) Configuración y clave de sesión

- **load_secret_key()**: obtiene la clave para firmar las cookies de sesión. Prioridad: variable de entorno `SECRET_KEY` → archivo local `.secret_key` → genera una aleatoria (`secrets.token_hex(32)`) y la persiste. **No hay clave insegura hardcodeada.**
- `app.secret_key` y la ruta a la BD (`DB`) se definen aquí.

b) Flask-Login (autenticación)

- **class User(UserMixin)**: modelo mínimo de usuario (id + username) que Flask-Login necesita.
- **load_user(user_id)**: le dice a Flask-Login cómo recuperar un usuario desde la BD a partir del id de la cookie.
- **unauthorized()**: qué hacer cuando alguien no autenticado accede a algo protegido → responde **401** en rutas `/api/...` o **redirige a /login** en el resto.

c) Base de datos y migraciones

- **get_db()**: abre una conexión SQLite con `row_factory = Row` (permite acceder a las columnas por nombre, ej. `row["type"]`).
- **init_db()**: crea las tablas `users` y `sessions` si no existen y ejecuta **migraciones automáticas** para bases de datos antiguas:
 - añade `users.theme` y `users.accent` (preferencias de apariencia);
 - añade `sessions.user_id` (columna que faltaba en el esquema original y que hacía fallar los guardados).

d) Rutas de autenticación

Función	Ruta	Qué hace
<code>login_page()</code>	GET <code>/login</code>	Sirve <code>login.html</code> (o redirige a <code>/</code> si ya estás logueado)

Función	Ruta	Qué hace
<code>register()</code>	POST <code>/api/register</code>	Crea cuenta (valida usuario/contraseña, hashea, inicia sesión)
<code>login()</code>	POST <code>/api/login</code>	Verifica credenciales e inicia sesión
<code>logout()</code>	GET <code>/logout</code>	Cierra sesión y redirige a <code>/login</code>
<code>me()</code>	GET <code>/api/me</code>	Devuelve el usuario actual + su tema y acento
<code>save_preferences()</code>	POST <code>/api/preferences</code>	Guarda tema y color de acento (con validación)

e) Rutas de la aplicación (todas requieren login)

Función	Ruta	Qué hace
<code>index()</code>	GET <code>/</code>	Sirve <code>index.html</code> (la app)
<code>get_sessions()</code>	GET <code>/api/sessions</code>	Lista sesiones del usuario. Filtros: <code>?days</code> , <code>?type</code> , <code>?include_breaks=1</code>
<code>add_session()</code>	POST <code>/api/sessions</code>	Inserta una sesión (estudio o descanso)
<code>get_stats()</code>	GET <code>/api/stats</code>	Estadísticas calculadas en el servidor (alternativa)
<code>export_csv()</code>	GET <code>/api/export/csv</code>	Descarga el historial en CSV (con BOM para Excel)
<code>export_json()</code>	GET <code>/api/export/json</code>	Descarga el historial en JSON
<code>delete_all()</code>	DELETE <code>/api/sessions/all</code>	Borra todas las sesiones del usuario

Detalle importante: `get_sessions()` filtra los descansos (`mode='break'`) por defecto; hay que pasar `?include_breaks=1` para incluirlos (lo necesita el cálculo del Ratio de Descanso Activo). Casi todas las consultas llevan `WHERE user_id = ?` para **aislar los datos por usuario**.

`wsgi.py` — Punto de entrada para producción (10 líneas)

Archivo que **PythonAnywhere** (u otro servidor WSGI) busca para arrancar la app. Importa `app` e `init_db` de `app.py`, ejecuta la migración/creación de la BD al arrancar y expone la variable `application` (nombre que el servidor WSGI espera por convención).

En desarrollo no se usa: ahí se ejecuta `python app.py` directamente.

`ver_db.py` — Utilidad de inspección de la BD (21 líneas)

Script independiente para **mirar el contenido de `study.db`** desde la terminal sin abrir la app. Cuenta el total de registros y muestra los últimos 20 en una tabla formateada. Útil para depurar. Se ejecuta con `python ver_db.py`.

requirements.txt — Dependencias de Python

Lista mínima que instala `pip install -r requirements.txt`:

```
flask>=3.0.0
flask-login>=0.6.0
```

(Werkzeug y demás vienen como dependencias de Flask.)

study.db — Base de datos SQLite

Archivo binario único que contiene **todas** las tablas y datos. Se crea solo la primera vez que arranca la app. No se edita a mano (usa `ver_db.py` o la app). Ver el esquema en la sección 6.

.secret_key — Clave de sesión

Archivo de texto con la clave aleatoria que firma las cookies. Lo genera `load_secret_key()` si no existe. **No debe subirse a git** ni compartirse: quien la tenga puede falsificar sesiones.

README.md

Guía rápida orientada a *instalar* y *usar*: pasos de instalación, tabla de endpoints y esquema de la BD. Este `DOCUMENTACION.md` es la versión extendida y explicativa.

static/index.html — La aplicación completa (~1680 líneas)

Es el archivo más grande: contiene **HTML + CSS + JavaScript** en un solo documento. Se sirve en `/` (solo con login). Se divide conceptualmente en tres partes:

1) CSS (dentro de `<style>`)

- **Variables de tema** (`:root` y `[data-theme="..."]`): definen colores para los 4 temas (`dark`, `light`, `ocean`, `forest`). El color de acento (`--accent`) se inyecta por JS.
- **Glassmorphism**: tarjetas translúcidas con `blur`, formas de fondo animadas, sombras.
- Estilos de cada componente: timer, pestañas, tarjetas de métricas, gráficos, heatmap, selector de apariencia, etc.

2) HTML (dentro de `<body>`) — organizado en pestañas

- **Timer**: temporizador Pomodoro circular, configuración (min de trabajo/descanso/ciclos), campo de tipo de estudio **manual** con recomendaciones, y formulario de sesión manual.
- **Estadísticas**: tarjetas de métricas + gráficos (tendencia 7 días, por tipo, densidad por hora, radar, heatmap).
- **Registro**: tabla filtrable del historial + exportación.
- **Apariencia**: selector de tema y color de acento.

3) JavaScript (dentro del último `<script>`)

Agrupado por responsabilidad:

- **Temporizador:** `startTimer`, `pauseTimer`, `phaseComplete`, `renderClock`... Controla el ciclo Pomodoro y, al terminar cada fase, registra la sesión (estudio o descanso).
- **Persistencia y sincronización:** `logSession` (guarda local + POST al backend), `syncSessions` (merge idempotente por `ts` + backfill), `sessionKey/sessionTime`.
- **Tipos y recomendaciones:** `renderTypeRecos`, `pickReco`, `updateCustomType` (los 3 tipos más usados como sugerencia).
- **Estadísticas y métricas:** `renderStats` y helpers `computeStreaks`, `computeIRS`, `computeRDA`, `studyMinutesByDate` (ver glosario en la sección 8).
- **Gráficos** (Chart.js): gráfico semanal de línea, barras por tipo, densidad de 24h, `renderRadar` (top-6 tipos vs meta equilibrada) y `renderHeatmap` (calendario estilo GitHub con intensidad por `color-mix(var(--accent))`).
- **Apariencia:** `applyTheme`, `applyAccent`, `selectTheme`, `selectAccent`, `savePreferences` (persisten en el backend).
- **Arranque:** al cargar, `fetch('/api/me')` aplica las preferencias del usuario y llama a `syncSessions()`.

Utilidades comunes de los gráficos

- `cssVar(name)`: lee una variable CSS del tema activo → los gráficos se adaptan al color.
- `makeBarAxisOptions()`: opciones de ejes/rejilla generadas según el tema.
- `hexToRgb(hex)`: convierte el acento a `r,g,b` para usar transparencias.

`static/login.html` — Registro e inicio de sesión (~433 líneas)

Página independiente (mismo estilo glassmorphism) con un formulario que alterna entre "Iniciar sesión" y "Crear cuenta". `handleSubmit()` envía las credenciales por `fetch` a `/api/login` o `/api/register` y, si todo va bien, redirige a `/`. Muestra mensajes de error/éxito en pantalla.

`venv/` y `__pycache__/`

- `venv/`: entorno virtual con las dependencias de Python instaladas de forma aislada. No se toca a mano; se activa antes de ejecutar.
- `__pycache__/`: bytecode compilado que Python genera automáticamente para acelerar la carga. Se puede borrar sin consecuencias.

6. Esquema de la base de datos

Tabla `users`

Columna	Tipo	Descripción
<code>id</code>	INTEGER	Clave primaria
<code>username</code>	TEXT	Nombre de usuario (único)

Columna	Tipo	Descripción
password	TEXT	Hash de la contraseña (Werkzeug)
theme	TEXT	Tema de la interfaz (dark por defecto)
accent	TEXT	Color de acento (#6366f1 por defecto)

Tabla sessions

Columna	Tipo	Descripción
id	INTEGER	Clave primaria
user_id	INTEGER	Usuario propietario (FK → users.id)
date	TEXT	Fecha YYYY-MM-DD
hour	INTEGER	Hora del día (0–23)
time	TEXT	Hora formateada HH:MM
minutes	INTEGER	Duración en minutos
type	TEXT	Tipo de estudio (libre)
mode	TEXT	pomodoro / manual / break
ts	TEXT	Timestamp ISO completo — clave de deduplicación en la sincronización

7. Flujos clave

Registro / login → login.html envía credenciales → Flask valida y crea la cookie de sesión → redirige a / → index.html carga.

Registrar una sesión → el timer termina una fase → logSession() guarda en localStorage y hace POST /api/sessions → el backend inserta la fila con user_id.

Sincronización al abrir → GET /api/me (aplica tema/acento) → syncSessions() hace GET /api/sessions?include_breaks=1 → merge por ts + backfill de lo que faltaba en el backend → re-render de la UI.

Cambiar apariencia → eliges tema/acento → applyTheme/applyAccent actualizan las variables CSS al vuelo → POST /api/preferences lo persiste en tu cuenta.

8. Glosario de métricas (calculadas en el cliente)

Métrica	Función	Definición
Racha	computeStreaks	Días consecutivos estudiando (actual desde hoy/ayer, y máxima histórica). Un día cuenta con ≥ 1 min

Métrica	Función	Definición
IRS (Índice de Regularidad Semanal)	<code>computeIRS</code>	% de los últimos 7 días con ≥ 20 min estudiados
RDA (Ratio de Descanso Activo)	<code>computeRDA</code>	Minutos de descanso $\div$ minutos de estudio (referencia ideal ~20% del método Pomodoro)
Enfoque Pomodoro	en <code>renderStats</code>	% del tiempo de estudio realizado en modo <code>pomodoro</code> (vs manual)
Heatmap de consistencia	<code>renderHeatmap</code>	Calendario de las últimas 26 semanas; intensidad de color por minutos/día
Balance temático	<code>renderRadar</code>	Radar de los 6 tipos más estudiados vs una meta equilibrada (promedio)

9. Cómo ejecutar

```
python -m venv venv          # crear entorno (una vez)
venv\Scripts\activate       # activar (Windows)
pip install -r requirements.txt
python app.py                # arranca en http://127.0.0.1:5000
```

Producción (PythonAnywhere): define la variable de entorno `SECRET_KEY` y usa `wsgi.py` como punto de entrada.

10. Notas y buenas prácticas

- Si algún día usas **git**, ignora: `.secret_key`, `study.db`, `venv/`, `__pycache__`.
- Las dependencias de frontend van por **CDN**; sin conexión, los gráficos e iconos no cargarán (la app base sigue funcionando).
- El heatmap usa `color-mix()`, soportado por navegadores modernos (Chrome/Edge/Firefox actuales).
- El servidor de desarrollo (`app.run(debug=True)`) **no** debe usarse en producción; ahí se usa el servidor WSGI del hosting.